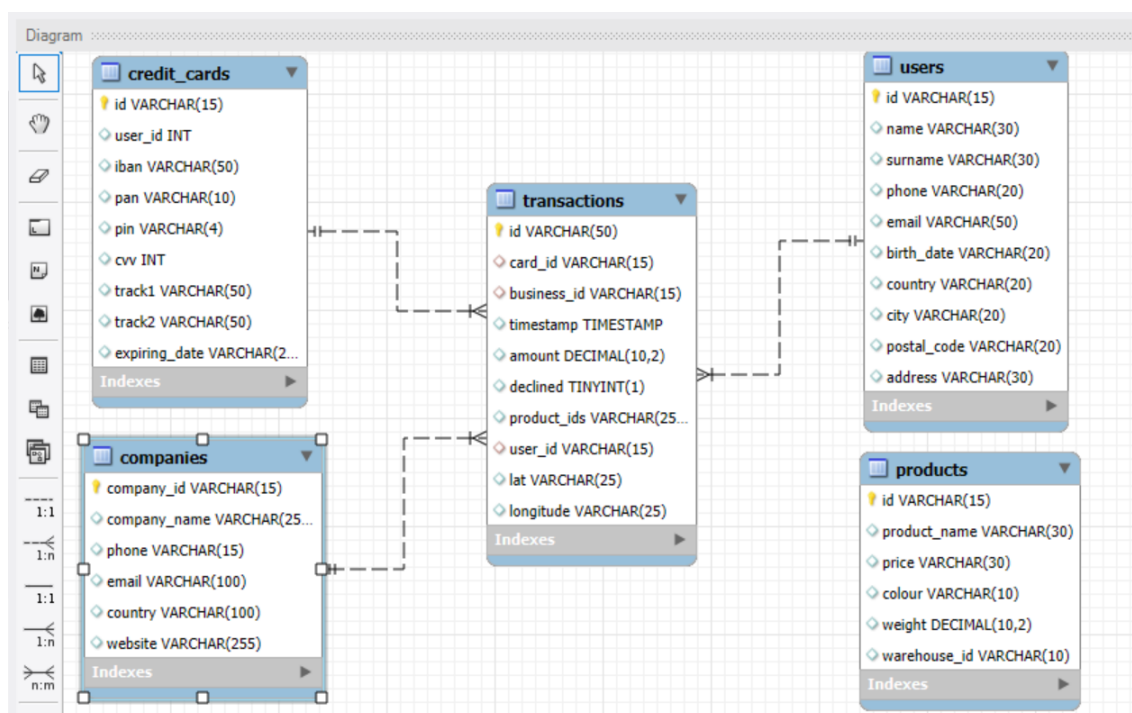
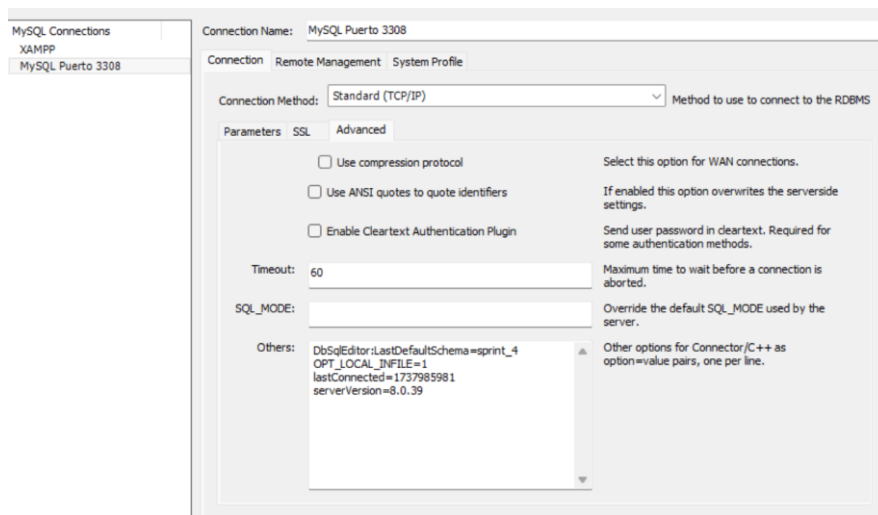


NIVEL 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes

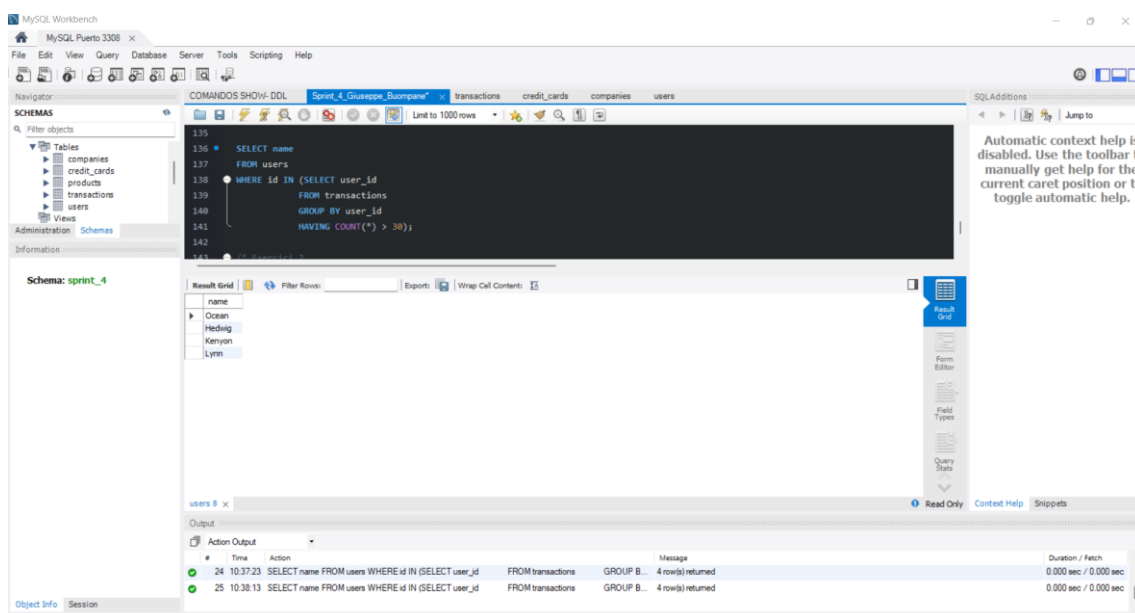


Aquí creamos la BBDD Sprint_4 y luego creamos tablas y acto seguido cargamos datos, por ultimo creamos transactions, cargamos datos y creamos las FK/constraints . Creamos 1 sola tabla users, cargando los datos de los 3 países ya que tienen los mismos campos/datos. Dependiendo de los separados de los CSV y el entorno en los que fueron creados, tenemos que ajustar el código de la carga. Configurar el workbench añadiendo en manage server connections, en el box others: OPT_LOCAL_INFILE =1 + ejecutar el comando SET GLOBAL local_infile=1 para permitir la carga local de los archivos en caso de que esté deshabilitada. La tabla products está creada pero no está relacionada aún con la BBDD.



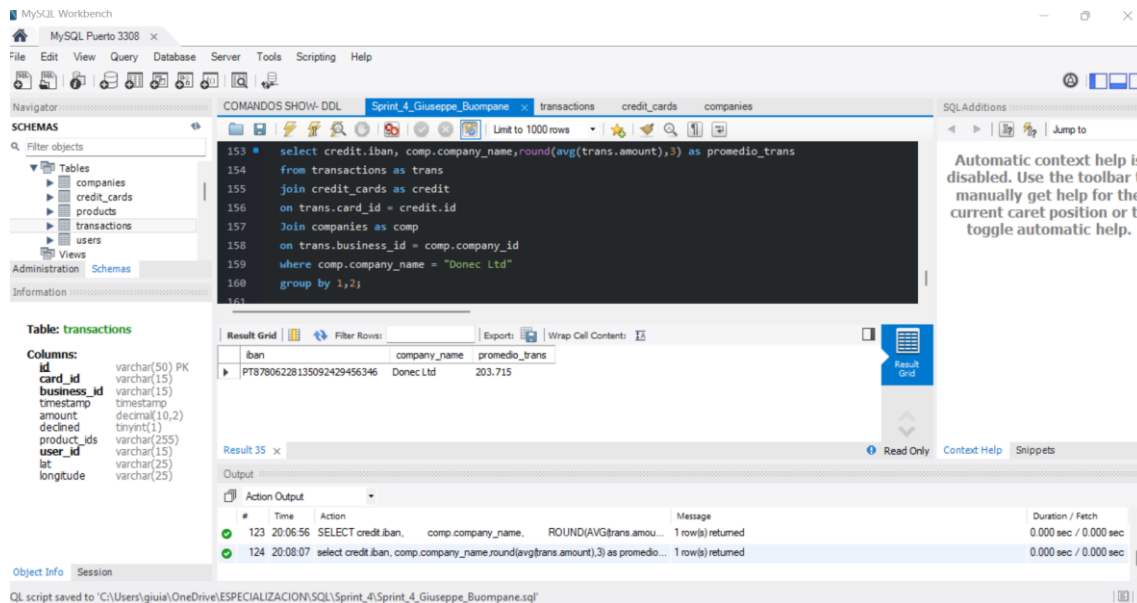
- Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 30 transaccions utilitzant almenys 2 taules.



- Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

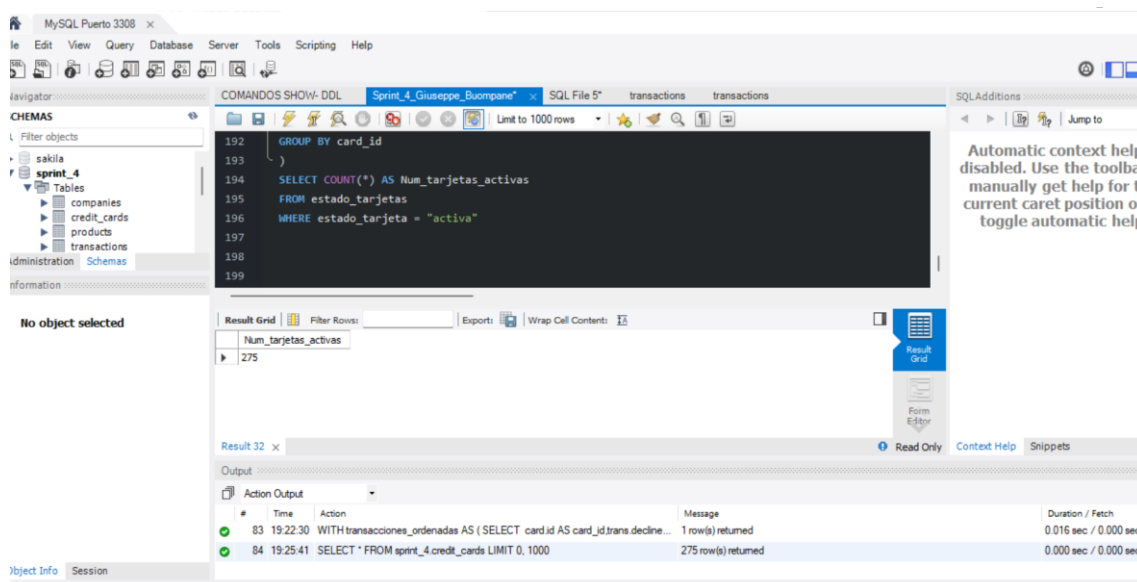


NIVEL 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les últimes tres transaccions van ser declinades i genera la següent consulta:

Exercici 1

Quantes targetes estan actives?



con el WITH creamos 2 tablas temporales sobre las que podemos trabajar, con la window function : row_number() crea una nueva columna y a cada fila le asigna un numero a partir de 1.

Partition by: divide los datos (las transacciones) por cada id, con el order by timestamp desc, obtenemos las transacciones ordenadas por la más reciente, de este modo sabemos que por cada grupo id, el numero 1,2,3 son las ultimas 3 transacciones. En la tabla estado_tarjeta, utilizamos un CASE para crear una nueva columna con condición para establecer el estado de cada tarjeta(id).

las tarjetas que tienen la suma = 3 (1+1+1) del campo declined y 3 del count de filas son las tarjetas inactivas, las demás, recogidas por el else , están inactivas.

NIVEL 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following commands:

```
287 CREATE TABLE IF NOT EXISTS prod_transactions (
288   id INT AUTO_INCREMENT PRIMARY KEY,
289   transaction_id VARCHAR(58),
290   product_id VARCHAR(15)
291 );
292 INSERT INTO prod_transactions (transaction_id, product_id)
293 SELECT
294   trans_id,
295   TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(trans.product_ids, ',', n.n), ',', -1)) AS product_id
```

The left sidebar shows the database schema with tables: companies, credit_cards, prod_transactions, products, transactions, and users. The 'prod_transactions' table is selected, and its structure is shown in the bottom left:

Column	Type
id	int AI PK
transaction_id	varchar(50)
product_id	varchar(15)
cantidad	int

The 'Result Grid' shows one row of data:

id	transaction_id	product_id	cantidad
2048	B4793303-EDF5-4F53-39CB-AA6A2877CDF8	37	1

The 'Output' pane at the bottom shows the execution log:

#	Time	Action	Message	Duration / Fetch
71	19:18:05	SELECT * FROM sprint_4.transactions LIMIT 0, 1000	588 row(s) returned	0.000 sec / 0.000 sec
72	19:40:10	select * from prod_transactions where transaction_id = 'B4793303-EDF5-4F53-...	1 row(s) returned	0.000 sec / 0.000 sec

Hemos generado una tabla intermedia prod_transactions para resolver la relación N:N entre transactions y productos. La tabla intermedia contiene un campo ID por cada registro que hemos generado con un autoincrement (identificador único por cada registro), un campo transaction_id y un campo product_id que refleja cada uno de los ids que hemos extraído del campo producto_ids de la tabla transacions.

Al crear la tabla, hemos estado barajando la idea de crear un campo 'cantidad', al final lo hemos añadido tras la creación de la tabla, como tarea adicional, especificando mis consideraciones y realizando una nueva consulta, la tarea en cuestión se encuentra detallada en el file sql.

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
259 FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE;
260
261 /*Exercici 1
262 Necessitem conèixer el nombre de vegades que s'ha venut cada producte*/
263
264 SELECT prod.id, prod.product_name, count(prodtra.product_id) prod_vendidos
265 FROM prod_transactions as prodtra
266 Join products as prod
267 on prodtra.product_id = prod.id
```

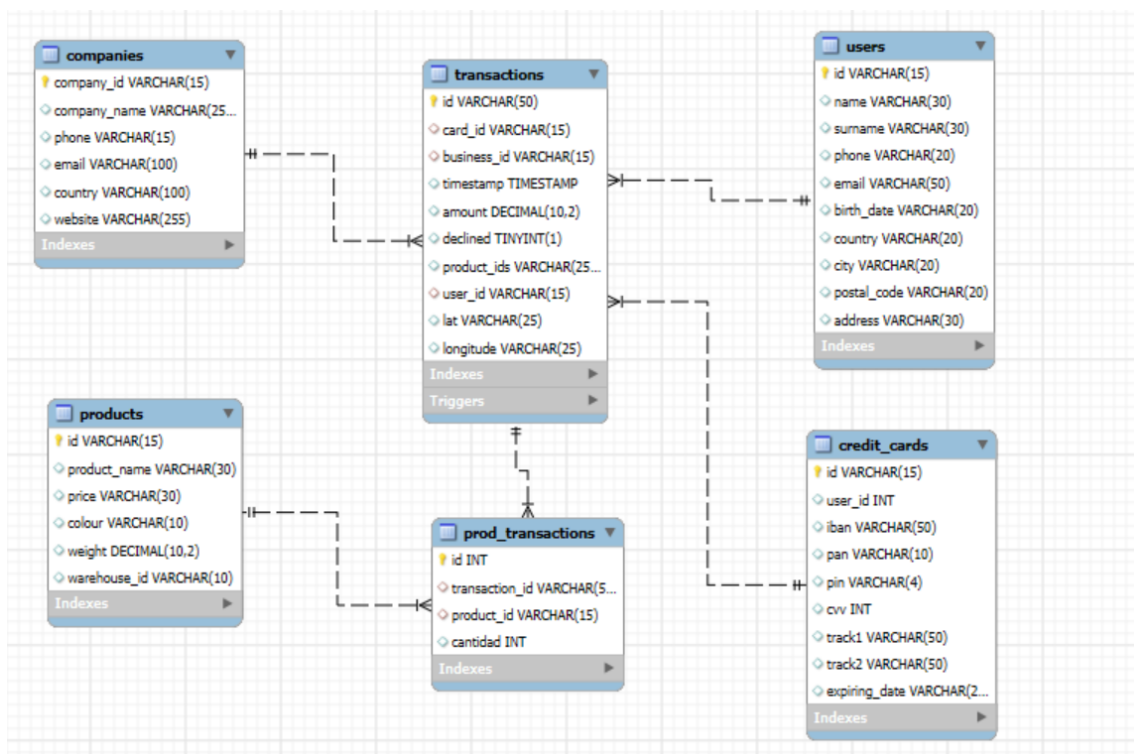
The Results grid shows the following data:

id	product_name	prod_vendidos
23	riverlands north	68
67	Winterfell	68
79	Direwolf riverlands the	66
2	Tarly Stark	65
43	duel	65

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
109	18.23.52	SELECT prod.id, prod.product_name, count(prodtra.product_id) prod_vendidos ...	25 row(s) returned	0.000 sec / 0.000 sec
110	18.23.58	SELECT prod.id, prod.product_name, count(prodtra.product_id) prod_vendidos ...	25 row(s) returned	0.016 sec / 0.000 sec

Diagrama final (modelo dimensional copo de nieve) y tareas adicionales



The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'actualiza_prod_trans' trigger under the 'users' schema. The main window shows the SQL script for creating the trigger 'actualiza_prod_transacion_AI' as an AFTER INSERT trigger on the 'transactions' table. The script includes comments in Spanish explaining the trigger's logic: it selects fields from 'prod_transaccion' and 'Transacciones' to update the 'prod_transactions' table. Below the script, the 'Result Grid' shows the execution of the trigger, with two rows of data returned. The 'Output' pane at the bottom shows the execution of the trigger, with two rows of data returned, indicating successful execution.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Trigger:
actualiza_prod_transacion_AI

Definition:
Event: INSERT
Timing: AFTER

Result Grid:

id	transaction_id	product_id	cantidad
2048	B4793303-EDF5-4F53-39CB-AA6A2877CDF8	37	1

Output:

#	Time	Action	Message	Duration / Fetch
77	20:04:34	select * from transactions where id= 'B4793303-EDF5-4F53-39CB-AA6A2877C...	1 row(s) returned	0.000 sec / 0.000 sec
78	20:04:40	select * from prod_transactions where transaction_id = 'B4793303-EDF5-4F53-...	1 row(s) returned	0.000 sec / 0.000 sec

Hemos creado un trigger para que también se actualice automáticamente prod_transactions cada vez que se inserte un nuevo registro en Transactions. Insertamos un registro y verificamos ejecución (detalles con código en el file sql).